


```

interface AxisState {
    axis_id: string;

    /** Current posterior distribution over the axis */
    posterior: AxisPosterior;

    /** Shannon entropy of the posterior:  $H = -\sum(p * \log_2(p))$  for  $p > 0$  */
    entropy: number;

    /** Maximum possible entropy for this axis ( $\log_2(11) = 3.459$  for uniform) */
    max_entropy: number;

    /** Point estimate:  $E[\text{posterior}] = \sum(\text{bin\_value} * p_{\text{bin}})$  */
    point_estimate: number;

    /** Confidence derived from posterior concentration:  $1 - (\text{entropy} / \text{max\_entropy})$  */
    confidence: number;

    /** Importance score from user signals (0-10). Null if never discussed. */
    importance: number | null;

    /** Which turn last produced a signal update for this axis */
    last_turn_updated: number;

    /** How many turns have directly probed this axis (asked about it) */
    turns_probed: number;

    /** Soft signals received via spillover (not directly probed) */
    spillover_signal_count: number;

    /** Coverage status for the matching formula */
    coverage_status: 'uncovered' | 'soft_only' | 'partial' | 'covered';

    /** Evidence quotes accumulated for this axis */
    evidence_quotes: string[];
}

/** Session-level fields */
interface ConversationRouterState {
    session_id: string;
    ballot_id: string;

    /** All 16 axes */
    axes: Record<string, AxisState>;

    /** Global turn counter (user turns only) */
    turn_count: number;

    /** Maximum turns before forced session completion */
    max_turns: number;

    /** Axes that appear on the user's actual ballot items */
    ballot_relevant_axes: Set<string>;
}

```

```

/** Axes that have been directly probed (question was about this axis) */
probed_axes: string[];

/** History of which axis was targeted each turn */
routing_history: Array<{
  turn: number;
  target_axis: string;
  spillover_axes: string[];
  entropy_before: number;
  entropy_after: number;
}>;

/** Session-level entropy: sum of per-axis entropies weighted by relevance */
session_entropy: number;

/** Estimated turns remaining (computed by stopping policy) */
estimated_turns_remaining: number;

/** Whether the router has determined we have enough signal */
ready_for_ballot: boolean;
}

```

2.2 Posterior Initialization

All axes start with a uniform prior (maximum ignorance):

```

const UNIFORM_PRIOR: AxisPosterior = {
  bins: [1/11, 1/11, 1/11, 1/11, 1/11, 1/11, 1/11, 1/11, 1/11, 1/11, 1/11]
};

const MAX_ENTROPY = Math.log2(11); // 3.459 bits

function initializeAxisState(axis_id: string): AxisState {
  return {
    axis_id,
    posterior: { ...UNIFORM_PRIOR, bins: [...UNIFORM_PRIOR.bins] },
    entropy: MAX_ENTROPY,
    max_entropy: MAX_ENTROPY,
    point_estimate: 5.0,
    confidence: 0.0,
    importance: null,
    last_turn_updated: -1,
    turns_probed: 0,
    spillover_signal_count: 0,
    coverage_status: 'uncovered',
    evidence_quotes: [],
  };
}

```

2.3 Posterior Update from a Signal

When a `ValueSignal` arrives (direction `d`, confidence `c`), we update the axis posterior using a Gaussian likelihood centered at `d` with variance inversely proportional to `c`:

```
function updatePosterior(
  axis: AxisState,
  signal_direction: number, // 0-10
  signal_confidence: number, // 0-1
  signal_importance: number, // 0-10
  is_spillover: boolean,
  turn: number,
  quote: string
): AxisState {
  const updated = structuredClone(axis);

  // Likelihood: Gaussian centered at signal_direction
  // Variance decreases as confidence increases (higher confidence = sharper peak)
  // sigma ranges from 4.0 (confidence=0.1) to 0.8 (confidence=0.9)
  const sigma = 4.0 - (signal_confidence * 3.5);
  const sigma_sq = sigma * sigma;

  // Compute unnormalized posterior = prior * likelihood for each bin
  const unnormalized: number[] = new Array(11);
  let total = 0;
  for (let bin = 0; bin <= 10; bin++) {
    const diff = bin - signal_direction;
    const likelihood = Math.exp(-(diff * diff) / (2 * sigma_sq));

    // Spillover signals get a flatter likelihood (less informative)
    const effective_likelihood = is_spillover
      ? 0.3 + 0.7 * likelihood // blend toward uniform
      : likelihood;

    unnormalized[bin] = updated.posterior.bins[bin] * effective_likelihood;
    total += unnormalized[bin];
  }

  // Normalize
  if (total > 0) {
    for (let bin = 0; bin <= 10; bin++) {
      updated.posterior.bins[bin] = unnormalized[bin] / total;
    }
  }

  // Recompute derived quantities
  updated.entropy = computeEntropy(updated.posterior);
  updated.point_estimate = computeExpectedValue(updated.posterior);
  updated.confidence = 1 - (updated.entropy / updated.max_entropy);

  // Update importance (weighted average if existing)
  if (updated.importance === null) {
    updated.importance = signal_importance;
  } else {
    updated.importance = (updated.importance + signal_importance) / 2;
  }
}
```

```

}

// Bookkeeping
updated.last_turn_updated = turn;
if (is_spillover) {
    updated.spillover_signal_count++;
}
if (quote) {
    updated.evidence_quotes.push(quote);
}

// Update coverage status
if (updated.confidence >= 0.6) {
    updated.coverage_status = 'covered';
} else if (updated.confidence >= 0.3) {
    updated.coverage_status = 'partial';
} else if (updated.spillover_signal_count > 0 || updated.confidence > 0.05) {
    updated.coverage_status = 'soft_only';
}

return updated;
}

function computeEntropy(posterior: AxisPosterior): number {
    let h = 0;
    for (const p of posterior.bins) {
        if (p > 1e-10) {
            h -= p * Math.log2(p);
        }
    }
    return h;
}

function computeExpectedValue(posterior: AxisPosterior): number {
    let ev = 0;
    for (let bin = 0; bin <= 10; bin++) {
        ev += bin * posterior.bins[bin];
    }
    return ev;
}

```

3. Multi-Axis Yield Extraction (signal_spillover)

3.1 Design Rationale

The current system scopes extraction to `getRelevantDomainAxes(domainId)`. A user statement like "I support Medicare for All and think we should tax the rich to pay for it" currently only extracts signals for whichever domain is active. The spillover extractor runs after the primary extraction and detects implicit signals for ALL 16 axes.

3.2 Cross-Axis Implication Graph

Certain axis positions have known correlations. We encode these as a static graph used to generate candidate spillover signals, which are then confirmed or rejected by the LLM.

```
/**
 * Cross-axis implication edges. When a strong signal is detected on
 * source_axis in a given direction range, it implies a soft signal
 * on target_axis.
 *
 * These are NOT hard rules. They are priors that the extraction LLM
 * confirms or rejects. The implied_confidence is capped to prevent
 * spillover from dominating direct signals.
 */
interface SpilloverEdge {
  source_axis: string;
  source_direction_range: [number, number]; // [low, high] on 0-10 scale
  target_axis: string;
  implied_direction: number;
  implied_confidence_cap: number; // max confidence for the spillover
  rationale: string;
}

const SPILLOVER_GRAPH: SpilloverEdge[] = [
  // Medicare for All implies coverage model AND cost control AND tax structure
  {
    source_axis: 'health_coverage_model',
    source_direction_range: [0, 3],
    target_axis: 'econ_tax_structure',
    implied_direction: 2.5,
    implied_confidence_cap: 0.3,
    rationale: 'Universal coverage typically requires progressive taxation',
  },
  {
    source_axis: 'health_coverage_model',
    source_direction_range: [0, 3],
    target_axis: 'econ_investment',
    implied_direction: 2.0,
    implied_confidence_cap: 0.25,
    rationale: 'Universal healthcare implies comfort with public spending',
  },
  {
    source_axis: 'health_coverage_model',
    source_direction_range: [0, 3],
    target_axis: 'health_cost_control',
    implied_direction: 2.0,
    implied_confidence_cap: 0.35,
    rationale: 'Single-payer advocates typically favor government price controls',
  },
  // Strong climate ambition implies energy portfolio preference
  {
    source_axis: 'climate_ambition',
    source_direction_range: [0, 3],
    target_axis: 'climate_energy_portfolio',
  },
]
```

```

    implied_direction: 2.0,
    implied_confidence_cap: 0.3,
    rationale: 'Aggressive climate action usually implies renewables priority',
  },
  {
    source_axis: 'climate_ambition',
    source_direction_range: [0, 3],
    target_axis: 'climate_permitting',
    implied_direction: 2.5,
    implied_confidence_cap: 0.2,
    rationale: 'Climate urgency may imply desire for faster green permitting',
  },

  // Gun regulation stance implies policing stance (weak)
  {
    source_axis: 'justice_firearms',
    source_direction_range: [0, 3],
    target_axis: 'justice_policing_accountability',
    implied_direction: 3.0,
    implied_confidence_cap: 0.2,
    rationale: 'Gun regulation advocates often favor police reform (weak link)',
  },

  // Transit priority and zoning are correlated
  {
    source_axis: 'housing_transport_priority',
    source_direction_range: [0, 3],
    target_axis: 'housing_supply_zoning',
    implied_direction: 3.0,
    implied_confidence_cap: 0.25,
    rationale: 'Transit investment advocates often support density/zoning reform',
  },

  // School choice and safety net are weakly anti-correlated
  {
    source_axis: 'econ_school_choice',
    source_direction_range: [7, 10],
    target_axis: 'econ_safetynet',
    implied_direction: 7.0,
    implied_confidence_cap: 0.15,
    rationale: 'School choice proponents tend toward conditional safety nets
(weak)',
  },

  // Market-based coverage implies less public investment preference
  {
    source_axis: 'health_coverage_model',
    source_direction_range: [7, 10],
    target_axis: 'econ_investment',
    implied_direction: 7.5,
    implied_confidence_cap: 0.25,
    rationale: 'Market-based healthcare implies preference for lower public
spending',
  },

```

```
// Sentencing goals and policing accountability are correlated
{
  source_axis: 'justice_sentencing_goals',
  source_direction_range: [0, 3],
  target_axis: 'justice_policing_accountability',
  implied_direction: 2.5,
  implied_confidence_cap: 0.3,
  rationale: 'Rehabilitation focus and police reform often co-occur',
},
];
```

3.3 signal_spillover Pseudocode

```
interface SpilloverResult {
  axis_id: string;
  implied_score: number; // 0-10
  implied_confidence: number; // 0-1 (capped)
  source_axis: string;
  source_quote: string;
  rationale: string;
}

/**
 * Given the primary signals extracted this turn, detect implicit
 * signals for other axes via the spillover graph AND via LLM
 * re-extraction against the full 16-axis set.
 */
function signal_spillover(
  primary_signals: ValueSignal[],
  user_message: string,
  current_state: ConversationRouterState
): SpilloverResult[] {
  const results: SpilloverResult[] = [];
  const primary_axis_ids = new Set(primary_signals.map(s => s.axisId));

  // --- Phase 1: Graph-based spillover ---
  for (const signal of primary_signals) {
    for (const edge of SPILLOVER_GRAPH) {
      // Skip if this edge doesn't match the source signal
      if (edge.source_axis !== signal.axisId) continue;
      if (signal.direction < edge.source_direction_range[0]) continue;
      if (signal.direction > edge.source_direction_range[1]) continue;

      // Skip if the target axis already has a primary signal this turn
      if (primary_axis_ids.has(edge.target_axis)) continue;

      // Skip if target axis already has high confidence
      const target_state = current_state.axes[edge.target_axis];
      if (target_state && target_state.confidence >= 0.6) continue;

      // Scale implied confidence by the source signal's confidence
      const implied_confidence = Math.min(
```



```

        edge.implicit_confidence_cap,
        signal.confidence * edge.implicit_confidence_cap
    );

    // Only emit if the implied confidence would actually be informative
    if (implicit_confidence < 0.05) continue;

    results.push({
        axis_id: edge.target_axis,
        implied_score: edge.implicit_direction,
        implied_confidence,
        source_axis: signal.axisId,
        source_quote: signal.source,
        rationale: edge.rationale,
    });
}
}

// --- Phase 2: LLM re-scan for cross-domain signals ---
// Only run if the user message is substantive (>20 chars) and
// there are uncovered axes that might benefit.
//
// This is implemented as an additional extraction pass (Pass 2b)
// scoped to axes NOT in the current domain. See Section 6 for
// integration into the two-pass pipeline.
//
// The LLM pass is optional and can be skipped under latency
// pressure. Graph-based spillover alone provides ~70% of the
// cross-domain signal capture.

return results;
}

```

3.4 Example: Multi-Axis Extraction from a Single Utterance

User says: "I support Medicare for All and think we should tax the rich to pay for it. Also, my neighborhood just got rezoned for high-density and I'm actually glad about it."

Primary extraction (current domain: healthcare):

axis_id	direction	confidence	source
health_coverage_model	1.0	0.85	"I support Medicare for All"
health_cost_control	2.0	0.55	"we should tax the rich to pay for it"

Spillover detection:

axis_id	implied_score	implied_confidence	source_axis	rationale
---------	---------------	--------------------	-------------	-----------

axis_id	implied_score	implied_confidence	source_axis	rationale
econ_tax_structure	2.5	0.25	health_coverage_model	Progressive taxation implied by M4A support
econ_investment	2.0	0.21	health_coverage_model	Comfort with public spending
housing_supply_zoning	2.0	0.40	(LLM re-scan)	Direct statement about density support

Key observation: The housing signal is NOT in the spillover graph — it was caught by the LLM re-scan (Pass 2b). The user explicitly said they're glad about high-density rezoning. Without cross-domain extraction, this signal would be lost until the housing domain comes up (if ever, under entropy routing).

Posterior updates after this turn:

```
health_coverage_model: H = 1.12 bits (was 3.46) – strong signal
health_cost_control:   H = 2.41 bits (was 3.46) – moderate signal
econ_tax_structure:    H = 3.01 bits (was 3.46) – soft spillover
econ_investment:       H = 3.10 bits (was 3.46) – soft spillover
housing_supply_zoning: H = 2.30 bits (was 3.46) – direct statement, caught by re-scan
```

The router now knows it can deprioritize `health_coverage_model` (low entropy) and potentially skip asking about zoning entirely if one more housing signal comes in.

4. Next-Axis Routing Policy (select_next_axis_conversational)

4.1 Design

The router selects the next axis to probe after each turn. It must balance four objectives:

- 1. **Entropy reduction** — Probe axes where we know the least.
- 2. **Ballot relevance** — Axes tied to actual ballot items matter more.
- 3. **Recency penalty** — Don't re-probe an axis we just asked about.
- 4. **Conversational coherence** — Avoid jarring topic switches; prefer axes in related domains.

4.2 Pseudocode

```

interface RoutingDecision {
  axis_id: string;
  routing_hint: string; // Injected into Pass 1 system prompt
  priority_score: number;
  reasoning: string;
}

/** Domain adjacency for conversational coherence (0 = same domain, 1 = related, 2
= distant) */
const DOMAIN_DISTANCE: Record<string, Record<string, number>> = {
  econ: { econ: 0, health: 1, housing: 1, justice: 2, climate: 2 },
  health: { econ: 1, health: 0, housing: 2, justice: 2, climate: 2 },
  housing: { econ: 1, health: 2, housing: 0, justice: 2, climate: 1 },
  justice: { econ: 2, health: 2, housing: 2, justice: 0, climate: 2 },
  climate: { econ: 2, health: 2, housing: 1, justice: 2, climate: 0 },
};

/** Map axis ID to its domain */
function axisToDomain(axis_id: string): string {
  if (axis_id.startsWith('econ_')) return 'econ';
  if (axis_id.startsWith('health_')) return 'health';
  if (axis_id.startsWith('housing_')) return 'housing';
  if (axis_id.startsWith('justice_')) return 'justice';
  if (axis_id.startsWith('climate_')) return 'climate';
  return 'unknown';
}

function select_next_axis_conversational(
  state: ConversationRouterState,
  last_probed_axis: string | null
): RoutingDecision {
  const candidates: Array<{ axis_id: string; score: number; reason: string }> = [];
  const last_domain = last_probed_axis ? axisToDomain(last_probed_axis) : null;

  for (const [axis_id, axis] of Object.entries(state.axes)) {
    // Skip axes already at sufficient confidence
    if (axis.confidence >= 0.7 && axis.coverage_status === 'covered') {
      continue;
    }

    // --- Factor 1: Entropy (normalized to 0-1) ---
    // Higher entropy = higher priority
    const entropy_score = axis.entropy / axis.max_entropy;

    // --- Factor 2: Ballot relevance ---
    // Axes tied to user's ballot get 2x weight
    const relevance_multiplier = state.ballot_relevant_axes.has(axis_id) ? 2.0 :
1.0;

    // --- Factor 3: Recency penalty ---
    // Penalize axes probed in the last 2 turns
    const turns_since_probed = axis.last_turn_updated >= 0
      ? state.turn_count - axis.last_turn_updated
      : Infinity;

```

```

let recency_penalty: number;
if (turns_since_probed === 0) {
  recency_penalty = 0.1; // Just probed this turn – strong penalty
} else if (turns_since_probed === 1) {
  recency_penalty = 0.4; // Probed last turn
} else if (turns_since_probed === 2) {
  recency_penalty = 0.7;
} else {
  recency_penalty = 1.0; // No penalty
}

// --- Factor 4: Conversational coherence ---
// Prefer axes in the same or adjacent domains
let coherence_bonus = 1.0;
if (last_domain) {
  const this_domain = axisToDomain(axis_id);
  const distance = DOMAIN_DISTANCE[last_domain]?.[this_domain] ?? 2;
  coherence_bonus = distance === 0 ? 1.3
    : distance === 1 ? 1.1
    : 0.9;
}

// --- Factor 5: Marginal value of probing ---
// Axes with only spillover signals benefit more from direct probing
const probe_bonus = axis.coverage_status === 'soft_only' ? 1.4
  : axis.coverage_status === 'uncovered' ? 1.2
  : 1.0;

// --- Factor 6: Fatigue awareness ---
// After many probes on the same axis, diminishing returns
const probe_fatigue = Math.max(0.3, 1.0 - (axis.turns_probed * 0.2));

// Composite score
const score = entropy_score
  * relevance_multiplier
  * recency_penalty
  * coherence_bonus
  * probe_bonus
  * probe_fatigue;

const reason = `H=${axis.entropy.toFixed(2)}, rel=${relevance_multiplier}, ` +
  `recency=${recency_penalty.toFixed(1)},
coherence=${coherence_bonus.toFixed(1)}, ` +
  `probe_bonus=${probe_bonus.toFixed(1)}`;

candidates.push({ axis_id, score, reason });
}

// Sort by score descending
candidates.sort((a, b) => b.score - a.score);

// Take the top candidate
const winner = candidates[0];
if (!winner) {
  // All axes covered – signal ready for ballot

```

```

    return {
        axis_id: '',
        routing_hint: 'All axes have sufficient coverage. Wrap up the conversation.',
        priority_score: 0,
        reasoning: 'No axes need further probing.',
    };
}

// Build a routing hint for the LLM response prompt
const routing_hint = buildRoutingHint(winner.axis_id, state);

return {
    axis_id: winner.axis_id,
    routing_hint,
    priority_score: winner.score,
    reasoning: winner.reason,
};
}

/**
 * Build a natural-language hint injected into the Pass 1 system prompt.
 * This steers the conversational LLM toward the target axis WITHOUT
 * revealing system internals.
 */
function buildRoutingHint(axis_id: string, state: ConversationRouterState): string
{
    const axis = state.axes[axis_id];

    // Topic hints per axis (natural language, no jargon)
    const AXIS_TOPIC_HINTS: Record<string, string> = {
        econ_safetynet: 'what kind of help should be available when people fall on hard
times',
        econ_investment: 'whether we should invest more in public services even if it
means higher taxes',
        econ_school_choice: 'how schools should work – public schools, charters,
vouchers',
        econ_tax_structure: 'who should pay more or less in taxes',
        health_coverage_model: 'how health insurance should work – government-run,
private, or a mix',
        health_cost_control: 'how to bring down the cost of healthcare',
        health_public_health: 'rules around public health – vaccines, mandates,
individual choice',
        housing_supply_zoning: 'building more housing vs. keeping neighborhoods the way
they are',
        housing_affordability_tools: 'how to make housing more affordable – subsidies,
rent control, or market solutions',
        housing_transport_priority: 'public transit vs. roads and car infrastructure',
        justice_policing_accountability: 'police reform and oversight',
        justice_sentencing_goals: 'what the justice system should focus on –
rehabilitation or punishment',
        justice_firearms: 'gun laws and regulations',
        climate_ambition: 'how urgently we should act on climate change',
        climate_energy_portfolio: 'where our energy should come from – renewables,
fossil fuels, or a mix',
        climate_permitting: 'how fast we should approve new energy and building

```

```

projects',
};

const topic = AXIS_TOPIC_HINTS[axis_id] || 'the next topic';

// Adapt the hint based on whether we have any signal
if (axis.coverage_status === 'uncovered') {
    return `Steer the conversation toward ${topic}. This is a new topic – start
with a relatable scenario.`;
} else if (axis.coverage_status === 'soft_only') {
    return `We have a faint signal about ${topic} from earlier. Ask a direct but
friendly question to confirm or refine their view.`;
} else if (axis.coverage_status === 'partial') {
    return `We have a partial read on ${topic}. Ask a targeted follow-up to get
more clarity – maybe a concrete tradeoff or scenario.`;
} else {
    return `We have a decent read on ${topic} but could use one more signal. Keep
it brief.`;
}
}

```

5. Dynamic Stopping Policy (should_move_on)

5.1 Design

The stopping policy operates at two levels:

- **Per-axis:** Should we stop probing a specific axis?
- **Per-session:** Should we end the warmup entirely?

Both use entropy as the primary criterion, augmented by fatigue and marginal gain estimates.

5.2 Pseudocode

```

interface StoppingDecision {
    should_stop: boolean;
    reason: string;
    /** If not stopping, estimated info gain of one more turn */
    marginal_gain_estimate: number;
}

// =====
// Per-Axis Stopping
// =====

function should_move_on_axis(
    axis: AxisState,
    state: ConversationRouterState
): StoppingDecision {
    // --- Threshold 1: Posterior entropy ---
    // Entropy below 1.5 bits means the posterior is concentrated
}

```

```

// enough for useful matching (out of max 3.459 bits).
const ENTROPY_THRESHOLD = 1.5;

if (axis.entropy < ENTROPY_THRESHOLD) {
  return {
    should_stop: true,
    reason: `Entropy ${axis.entropy.toFixed(2)} < ${ENTROPY_THRESHOLD} - axis
resolved`,
    marginal_gain_estimate: 0,
  };
}

// --- Threshold 2: Confidence floor ---
// Even without the entropy check, confidence >= 0.65 is sufficient
// for the matching formula (matches the structured path's threshold).
if (axis.confidence >= 0.65) {
  return {
    should_stop: true,
    reason: `Confidence ${axis.confidence.toFixed(2)} >= 0.65 - sufficient for
matching`,
    marginal_gain_estimate: 0,
  };
}

// --- Threshold 3: Diminishing marginal gain ---
// Estimate how much entropy an additional probe would reduce.
// Use the average entropy reduction per probe from history.
const avg_entropy_reduction = estimateAvgEntropyReduction(axis, state);

// If expected gain < 0.3 bits and we've probed at least twice, stop
if (avg_entropy_reduction < 0.3 && axis.turns_probed >= 2) {
  return {
    should_stop: true,
    reason: `Marginal gain ${avg_entropy_reduction.toFixed(2)} bits < 0.3 after
${axis.turns_probed} probes`,
    marginal_gain_estimate: avg_entropy_reduction,
  };
}

// --- Threshold 4: User fatigue ---
// If we've probed this axis 4+ times, stop regardless
if (axis.turns_probed >= 4) {
  return {
    should_stop: true,
    reason: `Probed ${axis.turns_probed} times - fatigue ceiling`,
    marginal_gain_estimate: avg_entropy_reduction,
  };
}

// --- Threshold 5: Non-ballot axis with any signal ---
// Axes not on the user's ballot need less precision
if (!state.ballot_relevant_axes.has(axis.axis_id) && axis.confidence >= 0.3) {
  return {
    should_stop: true,
    reason: `Non-ballot axis with confidence ${axis.confidence.toFixed(2)} - good

```

```

enough`,
    marginal_gain_estimate: avg_entropy_reduction,
  };
}

return {
  should_stop: false,
  reason: `Entropy ${axis.entropy.toFixed(2)}, confidence
${axis.confidence.toFixed(2)} - more signal needed`,
  marginal_gain_estimate: avg_entropy_reduction,
};
}

function estimateAvgEntropyReduction(
  axis: AxisState,
  state: ConversationRouterState
): number {
  // Look at the routing history for this axis
  const axis_history = state.routing_history.filter(
    h => h.target_axis === axis.axis_id || h.spillover_axes.includes(axis.axis_id)
  );

  if (axis_history.length === 0) {
    // No history - assume average reduction of 0.8 bits (optimistic prior)
    return 0.8;
  }

  const reductions = axis_history.map(h => h.entropy_before - h.entropy_after);
  const avg = reductions.reduce((a, b) => a + b, 0) / reductions.length;

  // Apply decay: later probes tend to yield less
  const decay = Math.pow(0.7, axis.turns_probed);
  return Math.max(0, avg * decay);
}

// =====
// Per-Session Stopping
// =====

function should_end_session(state: ConversationRouterState): StoppingDecision {
  // --- Hard ceiling ---
  if (state.turn_count >= state.max_turns) {
    return {
      should_stop: true,
      reason: `Turn limit reached (${state.max_turns})`,
      marginal_gain_estimate: 0,
    };
  }

  // --- All ballot-relevant axes resolved ---
  const ballot_axes = [...state.ballot_relevant_axes];
  const all_ballot_resolved = ballot_axes.every(axis_id => {
    const axis = state.axes[axis_id];
    return axis && (axis.confidence >= 0.5 || axis.entropy < 2.0);
  });
}

```



```

if (all_ballot_resolved && state.turn_count >= 3) {
  return {
    should_stop: true,
    reason: 'All ballot-relevant axes have sufficient coverage',
    marginal_gain_estimate: 0,
  };
}

// --- Session entropy threshold ---
// Weighted entropy across ballot-relevant axes
const session_entropy = computeSessionEntropy(state);
// Threshold: average entropy per ballot axis < 1.8 bits
const avg_entropy = ballot_axes.length > 0
  ? session_entropy / ballot_axes.length
  : 0;

if (avg_entropy < 1.8 && state.turn_count >= 4) {
  return {
    should_stop: true,
    reason: `Avg ballot-axis entropy ${avg_entropy.toFixed(2)} < 1.8`,
    marginal_gain_estimate: 0,
  };
}

// --- Fatigue heuristic ---
// After 12 turns, the user is likely tiring. Lower the bar.
if (state.turn_count >= 12 && avg_entropy < 2.5) {
  return {
    should_stop: true,
    reason: `Fatigue threshold: turn ${state.turn_count}, avg entropy
    ${avg_entropy.toFixed(2)}`,
    marginal_gain_estimate: 0,
  };
}

// Estimate remaining
const remaining_entropy = ballot_axes.reduce((sum, aid) => {
  const ax = state.axes[aid];
  return sum + (ax ? Math.max(0, ax.entropy - 1.5) : 0);
}, 0);
const avg_gain_per_turn = 0.6; // conservative estimate
const est_turns = Math.ceil(remaining_entropy / avg_gain_per_turn);

return {
  should_stop: false,
  reason: `Session entropy ${session_entropy.toFixed(2)}, est. ${est_turns} turns
  remaining`,
  marginal_gain_estimate: avg_gain_per_turn,
};
}

function computeSessionEntropy(state: ConversationRouterState): number {
  let total = 0;
  for (const axis_id of state.ballot_relevant_axes) {

```

```

    const axis = state.axes[axis_id];
    if (axis) {
        total += axis.entropy;
    }
}
return total;
}

```

5.3 Threshold Summary Table

Criterion	Threshold	Scope	Notes
Axis entropy	< 1.5 bits	Per-axis	Out of 3.459 max. Concentrated posterior.
Axis confidence	>= 0.65	Per-axis	Matches structured path for comparison.
Marginal gain	< 0.3 bits after 2+ probes	Per-axis	Diminishing returns detected.
Axis probes	>= 4	Per-axis	Hard fatigue ceiling per axis.
Non-ballot confidence	>= 0.3	Per-axis	Lower bar for axes not on ballot.
Session avg entropy	< 1.8 bits/axis, 4+ turns	Session	Ballot axes resolved on average.
Turn hard ceiling	>= max_turns (default 20)	Session	Never exceeds this.
Fatigue	12+ turns, avg entropy < 2.5	Session	Relaxed threshold for tired users.

6. Session Entropy Dashboard (entropy_summary)

6.1 Pseudocode

```

interface AxisEntropySummary {
    axis_id: string;
    entropy: number;
    confidence: number;
    coverage_status: string;
    turns_probed: number;
    point_estimate: number;
    is_ballot_relevant: boolean;
}

interface SessionEntropySummary {
    /** Total entropy across all axes */
    total_entropy: number;
}

```

```

/** Total entropy across ballot-relevant axes only */
ballot_entropy: number;

/** Average entropy per ballot-relevant axis */
avg_ballot_entropy: number;

/** Per-axis breakdown, sorted by entropy descending */
per_axis: AxisEntropySummary[];

/** Axes with confidence >= 0.5 (ready for matching) */
ready_axes: string[];

/** Axes with entropy > 2.5 bits (need more probing) */
high_entropy_axes: string[];

/** Axes never probed or discussed */
uncovered_axes: string[];

/** Estimated turns remaining to reach session stopping threshold */
estimated_turns_remaining: number;

/** Total user turns so far */
turns_elapsed: number;

/** Fraction of ballot-relevant axes that are 'covered' */
ballot_coverage_ratio: number;
}

function entropy_summary(state: ConversationRouterState): SessionEntropySummary {
  const per_axis: AxisEntropySummary[] = [];
  let total_entropy = 0;
  let ballot_entropy = 0;
  let ballot_count = 0;
  let ballot_covered = 0;
  const ready: string[] = [];
  const high_entropy: string[] = [];
  const uncovered: string[] = [];

  for (const [axis_id, axis] of Object.entries(state.axes)) {
    const is_ballot = state.ballot_relevant_axes.has(axis_id);

    per_axis.push({
      axis_id,
      entropy: axis.entropy,
      confidence: axis.confidence,
      coverage_status: axis.coverage_status,
      turns_probed: axis.turns_probed,
      point_estimate: axis.point_estimate,
      is_ballot_relevant: is_ballot,
    });

    total_entropy += axis.entropy;
    if (is_ballot) {
      ballot_entropy += axis.entropy;
      ballot_count++;
    }
  }
}

```

```

    if (axis.coverage_status === 'covered') ballot_covered++;
  }

  if (axis.confidence >= 0.5) ready.push(axis_id);
  if (axis.entropy > 2.5) high_entropy.push(axis_id);
  if (axis.coverage_status === 'uncovered') uncovered.push(axis_id);
}

// Sort by entropy descending (highest uncertainty first)
per_axis.sort((a, b) => b.entropy - a.entropy);

// Estimate remaining turns
const remaining_ballot_entropy = Math.max(0, ballot_entropy - (1.8 *
ballot_count));
const avg_gain_per_turn = 0.6;
const est_remaining = ballot_count > 0
  ? Math.ceil(remaining_ballot_entropy / avg_gain_per_turn)
  : 0;

return {
  total_entropy,
  ballot_entropy,
  avg_ballot_entropy: ballot_count > 0 ? ballot_entropy / ballot_count : 0,
  per_axis,
  ready_axes: ready,
  high_entropy_axes: high_entropy,
  uncovered_axes: uncovered,
  estimated_turns_remaining: Math.min(est_remaining, state.max_turns -
state.turn_count),
  turns_elapsed: state.turn_count,
  ballot_coverage_ratio: ballot_count > 0 ? ballot_covered / ballot_count : 0,
};
}

```

6.2 Example Dashboard Output (after 4 turns)

Session Entropy Dashboard

=====

Turns elapsed: 4 / 20 max

Estimated remaining: 6 turns

Ballot coverage: 4/10 axes covered (40%)

Session entropy: 42.3 bits (of 55.3 max)

Ballot entropy: 28.1 bits (of 34.6 max)

Avg ballot entropy: 2.81 bits/axis

HIGH ENTROPY (need probing):

climate_permitting	3.46 bits	uncovered	ballot
justice_sentencing_goals	3.46 bits	uncovered	ballot
econ_school_choice	3.20 bits	soft_only	
housing_affordability	3.10 bits	soft_only	ballot

READY (sufficient signal):

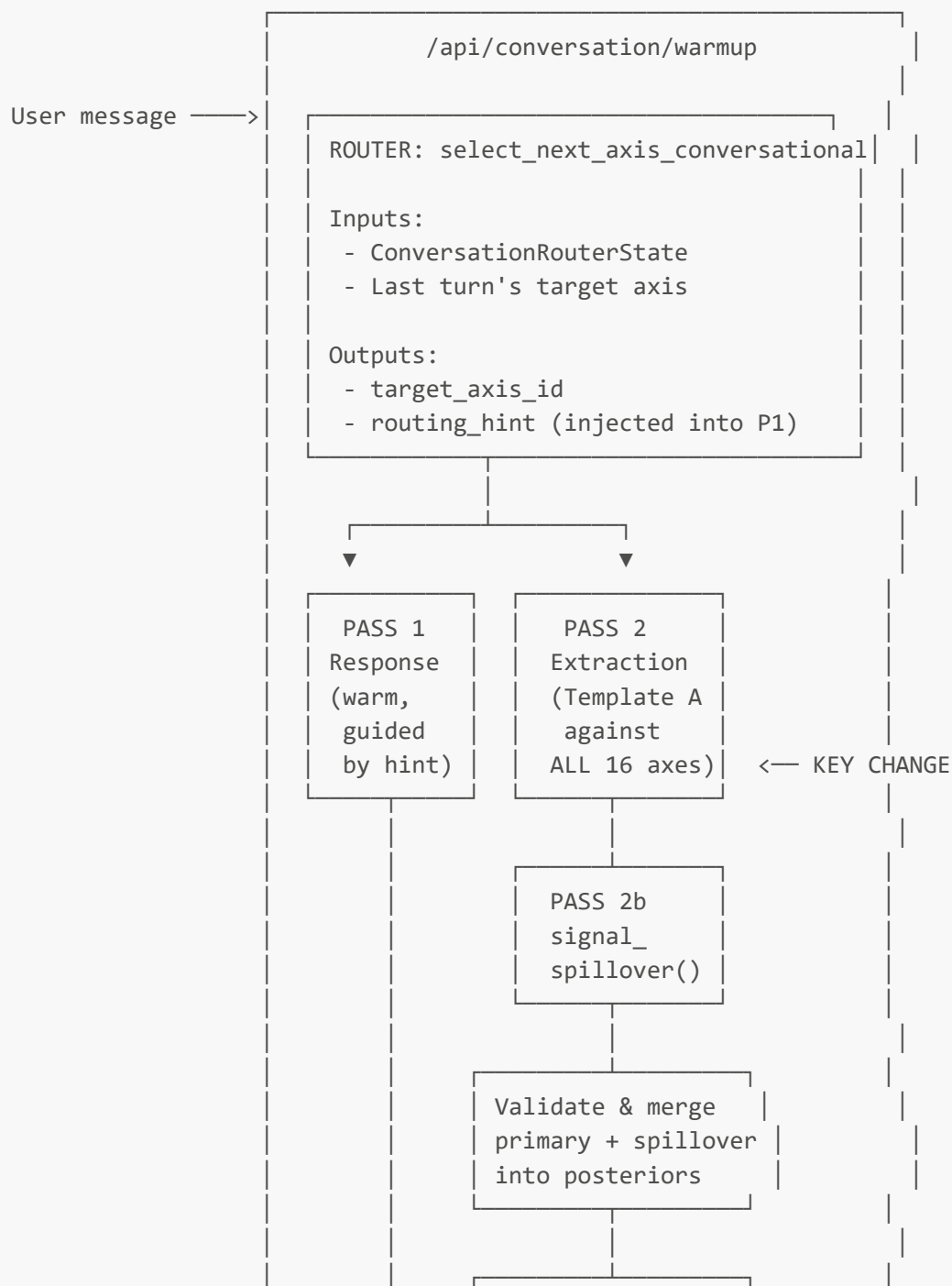
health_coverage_model	0.89 bits	covered	ballot
health_cost_control	1.44 bits	covered	ballot
econ_investment	1.80 bits	partial	ballot
housing_supply_zoning	1.55 bits	covered	ballot

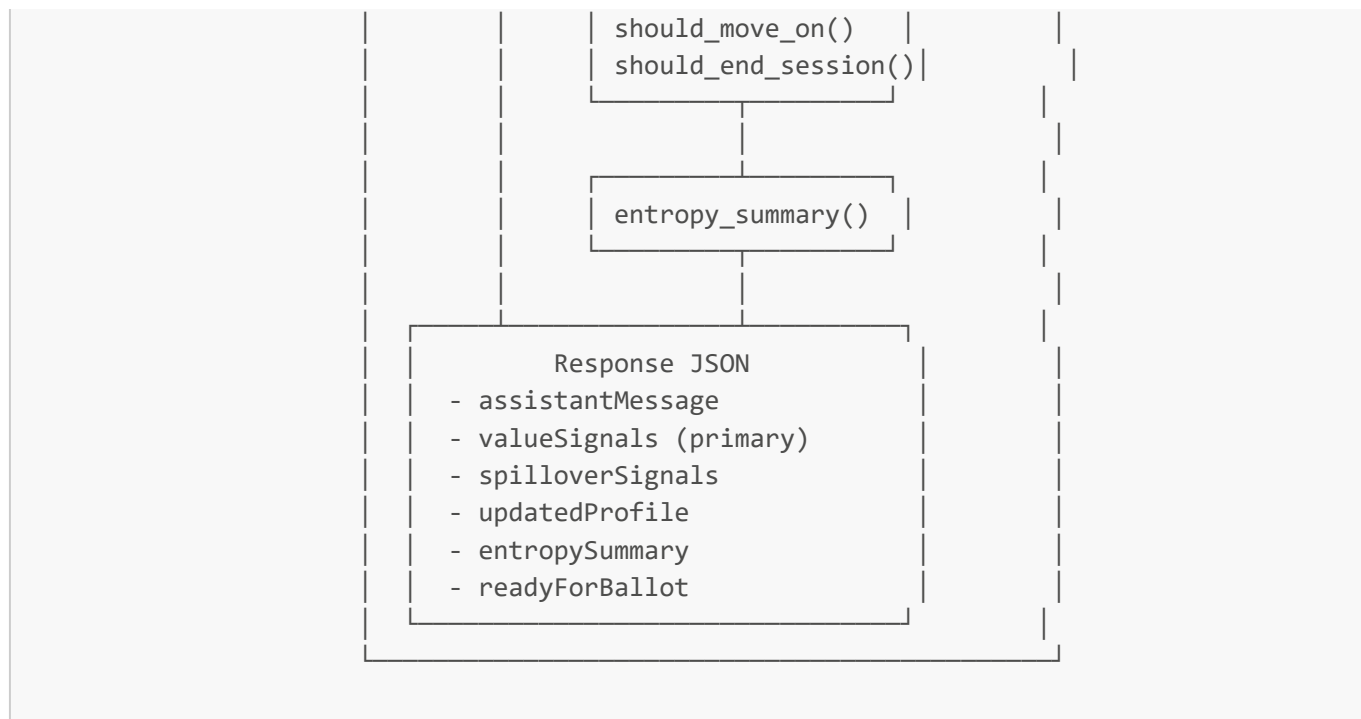
ROUTING DECISION: climate_permitting (score=3.21)

Reason: H=3.46, relevance=2.0, recency=1.0, coherence=0.9

7. Integration with the Two-Pass LLM Pipeline

7.1 Architecture Diagram





7.2 Key Changes from Current Implementation

Aspect	Current (domain-sequential)	Proposed (entropy-routed)
Axis selection	Fixed domain order, all axes in domain probed	Router selects highest-value axis each turn
Extraction scope	Current domain's axes only	All 16 axes, with spillover detection
Stopping	3 turns per domain, then advance	Per-axis entropy + marginal gain + fatigue
Cross-domain signals	Discarded	Captured via spillover graph + LLM re-scan
Profile representation	Point estimate + confidence	Full posterior distribution + entropy
Session state	<code>currentDomainIndex</code> + <code>domainTurnCount</code>	<code>ConversationRouterState</code> with per-axis posteriors
UI progress	"Domain 2 of 5" progress bar	Entropy-based progress (% of ballot axes covered)
Prompt injection	Domain label + turn count	<code>routing_hint</code> string with topic + coverage status

7.3 Migration Path

The router can be introduced incrementally:

Phase 1: Shadow mode. Run the router alongside the current domain-sequential logic. Log routing decisions without acting on them. Compare which axis the router would have chosen vs. what the sequential path actually probed. Measure wasted turns.

Phase 2: Hybrid mode. Use the router within each domain (replacing the fixed 3-turn count with entropy-based stopping), but keep the domain order fixed. This captures most of the per-axis stopping benefit without changing the conversational flow.

Phase 3: Full routing. Replace domain-sequential with pure entropy routing. The `routing_hint` steers the conversational LLM, and domains become an emergent property of which axes are selected rather than a prescribed order.

7.4 Pass 2 Modification for All-Axis Extraction

The extraction prompt (Template A) currently receives only the current domain's axes. Under the router, it receives all 16 axes with a two-tier structure:

```
=====
TARGET AXIS (primary extraction)
=====
```

```
[Full position reference for the target axis]
```

```
=====
SECONDARY AXES (extract ONLY if user explicitly mentioned)
=====
```

```
[Abbreviated reference for remaining 15 axes – name and pole labels only,
no full position descriptions. This keeps token count manageable.]
```

```
RULE: For the target axis, extract with full precision using the named
positions above. For secondary axes, ONLY extract a signal if the user's
words directly and unambiguously relate to that axis. Do NOT infer
secondary signals – that is handled separately by the spillover system.
```

This prevents the extraction LLM from hallucinating cross-domain signals while still capturing explicit cross-domain statements (like the "rezoned for high-density" example in Section 3.4).

8. Compatibility with the Matching Formula

The matching formula from the system specification is:

```
w_i = gw_i * cov_i * conf_i * estr_i
S    = sum(w_i * A_i) / sum(w_i)
```

Where:

- `gw_i` = global weight (axis importance to the user)
- `cov_i` = coverage status (0 if uncovered, 1 if covered, interpolated for partial)
- `conf_i` = confidence in the axis estimate
- `estr_i` = evidence strength

The router's `AxisState` maps directly:

Formula term	Router field	Derivation
<code>gw_i</code>	<code>axis.importance</code>	Normalized from 0-10 to a multiplier
<code>cov_i</code>	<code>axis.coverage_status</code>	<code>uncovered=0, soft_only=0.3, partial=0.6, covered=1.0</code>
<code>conf_i</code>	<code>axis.confidence</code>	<code>1 - (entropy / max_entropy)</code>
<code>estr_i</code>	Computed from signal sources	Direct probes > spillover > no evidence
<code>A_i</code>	<code>axis.point_estimate</code>	<code>E[posterior]</code> — expected value of the distribution

The posterior representation is strictly more informative than the current point estimate. The point estimate `A_i` is derived from it, and the confidence `conf_i` emerges naturally from the entropy rather than being an ad-hoc average of signal confidences.

9. Open Questions and Future Work

1. **Posterior vs. point estimate for matching.** The current formula uses point estimates. A more principled approach would propagate the full posterior through the match computation, yielding a distribution over match scores rather than a point. This would let the UI show "73% match (confident)" vs "73% match (uncertain)" — a material UX improvement. Deferred to a future iteration.
2. **Adaptive max_turns.** The current hard ceiling of 20 turns could adapt based on ballot complexity. A ballot with 3 items and 6 relevant axes needs fewer turns than one with 12 items and 14 relevant axes.
3. **User engagement signals.** Response length, response time, and explicit "I don't care" signals could feed into the fatigue model beyond simple turn counting.
4. **Spillover graph learning.** The static `SPILLOVER_GRAPH` could be learned from data as the system accumulates user sessions. Cross-axis correlations in real user profiles would refine the implied confidence caps.
5. **A/B testing framework.** The shadow mode (Section 7.3, Phase 1) naturally produces comparison data. We should instrument it to measure: turns to equivalent coverage, user satisfaction (exit survey), and match quality (compared to structured slider assessment).